

Guía para la Redacción de Requerimientos

SEGÚN ESTÁNDARES IEEE 830 / ISO/IEC 25010 Y README PARA
REPOSITORIOS EN GITHUB

Aplicado al desarrollo de proyectos de software con metodología ágil (Scrum)

PARTE I — Redacción de Requisitos (RF y RNF) según IEEE

1. ¿Qué es un Requisito de Software?

Un requisito de software es una declaración que describe lo que el sistema debe hacer (requisito funcional) o las restricciones bajo las cuales debe operar (requisito no funcional). La correcta redacción de requisitos es el fundamento sobre el que se construye cualquier proyecto de software profesional.

El estándar IEEE 830 (Especificación de Requisitos de Software) y la norma ISO/IEC 25010 establecen los criterios que debe cumplir un buen requisito:

Característica	Significado para el equipo
Correcto	Representa realmente lo que el cliente/usuario necesita.
Completo	No omite información. Describe el qué, con qué datos y qué resultado.
Verificable	Se puede diseñar una prueba (test) que demuestre su cumplimiento.
Consistente	No contradice a otro requisito del mismo documento.
No ambiguo	Solo admite una interpretación. Evita palabras como 'rápido', 'fácil', 'adecuado'.
Trazable	Puede identificarse de dónde viene (negocio) y hacia dónde va (historia de usuario, prueba).
Priorizable	Se puede clasificar por importancia para planificar sprints.
Modificable	Su estructura permite revisarlo sin reescribir todo el documento.

2. Requisitos Funcionales (RF)

Los Requisitos Funcionales describen las funciones, servicios o comportamientos que el sistema debe proveer. Responden a la pregunta: ¿qué debe hacer el sistema?

2.1 Estructura estándar (IEEE 830)

Plantilla para RF

RF-[ID] | Nombre descriptivo Descripción: El sistema debe [verbo en infinitivo] [objeto] [condición/contexto]. Prioridad: Alta / Media / Baja Fuente: [Actor/Stakeholder que lo solicitó] Criterio de verificación: [Condición que confirma su implementación correcta]

Reglas de redacción para RF:

- Usar siempre la voz activa: «El sistema debe registrar...» en lugar de «Los registros serán guardados...»
- Un requisito = una sola responsabilidad. No combinar dos funciones en un mismo enunciado.
- Evitar palabras ambiguas: rápido, intuitivo, moderno, apropiado, suficiente, etc.
- Indicar claramente el actor que desencadena la acción (usuario, administrador, sistema externo).

- Incluir siempre la condición de éxito observable (criterio de verificación).

2.2 Ejemplos comparativos — RF Mal redactados vs. Bien redactados

X MAL — RF-01 Registro de usuarios
El sistema debe permitir que los usuarios puedan registrarse fácilmente y rápido con sus datos personales.
✓ BIEN — RF-01 Registro de usuario con validación de datos
El sistema debe permitir al usuario no registrado crear una cuenta proporcionando nombre completo (máx. 100 caracteres), dirección de correo electrónico válida y contraseña de mínimo 8 caracteres (al menos una mayúscula, un número y un carácter especial). El sistema debe validar el formato del correo y la fortaleza de la contraseña en tiempo real, mostrando mensajes de error descriptivos ante cada incumplimiento. Al completar el registro correctamente, el sistema debe enviar un correo de confirmación al email ingresado en un tiempo máximo de 60 segundos.
X MAL — RF-02 Listado de productos
El sistema mostrará los productos disponibles.
✓ BIEN — RF-02 Listado de productos con filtrado y paginación
El sistema debe mostrar al usuario autenticado un listado paginado de productos (máximo 20 por página), con posibilidad de filtrar por categoría, rango de precio y disponibilidad de stock. Cada elemento debe mostrar: imagen, nombre, precio unitario y estado (disponible/sin stock). El tiempo de carga del listado no debe superar los 2 segundos bajo condiciones normales de uso.
X MAL — RF-03 Login
El sistema tiene login con usuario y contraseña.
✓ BIEN — RF-03 Autenticación de usuario mediante credenciales
El sistema debe permitir al usuario registrado iniciar sesión ingresando su correo electrónico y contraseña. Tras tres intentos fallidos consecutivos, el sistema debe bloquear el acceso por 15 minutos y notificar al usuario vía correo electrónico. El inicio de sesión exitoso debe redirigir al usuario al panel principal y establecer una sesión con token JWT de duración máxima de 8 horas.

3. Requisitos No Funcionales (RNF)

Los Requisitos No Funcionales (también llamados Requisitos de Calidad) describen las propiedades o restricciones del sistema: cómo debe ser, no qué debe hacer. Según la norma ISO/IEC 25010, se agrupan en las siguientes categorías:

Categoría ISO 25010	Descripción	Ejemplos de sub-características
Rendimiento / Eficiencia	Uso de recursos en relación con las condiciones establecidas.	Tiempo de respuesta, uso de CPU/memoria, throughput.
Seguridad	Protección de información y datos contra accesos no autorizados.	Autenticación, autorización, trazabilidad, cifrado.
Usabilidad	Facilidad de uso y satisfacción del usuario.	Comprensibilidad, aprendizaje, accesibilidad.
Confiabilidad	Capacidad de mantener el servicio bajo condiciones definidas.	Disponibilidad, tolerancia a fallos, recuperación.
Mantenibilidad	Facilidad para ser modificado y extendido.	Modularidad, reusabilidad, testabilidad.
Portabilidad	Facilidad de transferencia a otros entornos.	Adaptabilidad, instalabilidad, compatibilidad.
Compatibilidad	Coexistencia e interoperabilidad con otros sistemas.	Interoperabilidad, co-existencia.

3.1 Estructura estándar (IEEE 830)

Plantilla para RNF

RNF-[ID] | Nombre descriptivo | Categoría ISO 25010 Descripción: El sistema debe [propiedad medible] bajo [condición de contexto]. Métrica: [Indicador cuantificable. Ej: tiempo en ms, porcentaje de disponibilidad, etc.] Prioridad: Alta / Media / Baja Método de verificación: [Prueba de rendimiento / auditoría / revisión de código / etc.]

Reglas de redacción para RNF:

- Todo RNF debe ser MEDIBLE. Si no se puede medir, no es verificable.
- Especificar la condición de contexto: «bajo carga de 500 usuarios concurrentes» es muy diferente de «en condiciones normales».
- Indicar el método de verificación: ¿se valida con una prueba de carga? ¿Una auditoría de código? ¿Una herramienta de análisis estático?
- Evitar adjetivos vagos: «El sistema debe ser seguro» no es un RNF. «El sistema debe cifrar las contraseñas con bcrypt y un factor de costo mínimo de 10» sí lo es.
- Referenciar estándares conocidos cuando aplique: WCAG 2.1 (accesibilidad), OWASP Top 10 (seguridad), ISO 8601 (fechas), etc.

3.2 Ejemplos comparativos — RNF Mal redactados vs. Bien redactados

X MAL — RNF-01 Rendimiento Eficiencia
El sistema debe ser rápido y responder en tiempo real.

✓ BIEN — RNF-01 Tiempo de respuesta de endpoints REST | Eficiencia de rendimiento

El sistema debe responder al 95% de las solicitudes HTTP a los endpoints de la API REST en un tiempo inferior a 500 ms bajo una carga concurrente de hasta 100 usuarios simultáneos, medido en ambiente de staging con las mismas especificaciones de hardware del servidor de producción. Método de verificación: prueba de carga con Apache JMeter o k6.

✗ MAL — RNF-02 Seguridad | Seguridad

El sistema debe ser seguro y proteger los datos.

✓ BIEN — RNF-02 Protección de contraseñas almacenadas | Seguridad

El sistema debe almacenar todas las contraseñas de usuarios aplicando el algoritmo de hash bcrypt con un factor de costo mínimo de 12. Bajo ninguna circunstancia se deben almacenar contraseñas en texto plano ni con algoritmos reversibles (MD5, SHA-1 sin salt). Método de verificación: revisión de código y auditoría de la base de datos.

✗ MAL — RNF-03 Disponibilidad | Confiabilidad

El sistema debe estar disponible siempre.

✓ BIEN — RNF-03 Disponibilidad del servicio en producción | Confiabilidad

El sistema debe mantener una disponibilidad mínima del 99,5% mensual (máximo 3,6 horas de inactividad por mes), excluyendo ventanas de mantenimiento programadas notificadas con 48 horas de anticipación. Método de verificación: monitoreo continuo con herramienta de uptime (UptimeRobot, Datadog o equivalente) durante el período de aceptación.

4. Priorización con MoSCoW

La técnica MoSCoW permite priorizar los requisitos para planificar cuáles se implementan en cada Sprint:

Nivel	Significado	Cuándo usarlo
Must Have (M)	Obligatorio. Sin esto el sistema no funciona.	Funcionalidades del núcleo del negocio.
Should Have (S)	Importante pero no urgente. Debería estar en la entrega.	Valor alto, puede posponerse un sprint.
Could Have (C)	Deseable si hay tiempo y recursos disponibles.	Mejoras de experiencia de usuario, etc.
Won't Have (W)	No se implementará en esta iteración.	Ideas futuras, backlog de largo plazo.

5. Del Requisito Funcional a la Historia de Usuario

En Scrum, los Requisitos Funcionales se refinan y se expresan como Historias de Usuario (HU), que son la unidad de trabajo de cada Sprint. Una HU siempre tiene la misma estructura canónica:

Estructura de Historia de Usuario

Como [rol/actor], quiero [funcionalidad específica], para [beneficio / valor de negocio].

Ejemplo — trazabilidad RF → HU:

Origen	Contenido
RF-01 (Requisito Funcional)	El sistema debe permitir al usuario no registrado crear una cuenta proporcionando nombre completo, correo electrónico válido y contraseña de mínimo 8 caracteres con validación en tiempo real.
HU-01 (Historia de Usuario)	Como visitante no registrado, quiero crear una cuenta con mi nombre, email y contraseña, para acceder a las funcionalidades exclusivas de la plataforma.
Sprint	Sprint 1 — Prioridad: Must Have
Estimación	5 Story Points

5.1 Criterios de Aceptación

Cada Historia de Usuario debe tener Criterios de Aceptación (CA): condiciones verificables que confirman que la HU está completa y funciona correctamente. Se redactan en formato Gherkin (Given / When / Then):

Formato Gherkin para Criterios de Aceptación

Escenario: [Nombre descriptivo del caso] DADO QUE [contexto / precondition]
CUANDO [acción del usuario] ENTONCES [resultado esperado del sistema]

Criterios de Aceptación para HU-01 (Registro de usuario):

CA	Escenario	Dado que	Cuando	Entonces
CA-01	Registro exitoso	El visitante ingresa datos válidos	completa el formulario y presiona 'Registrarse'	el sistema crea la cuenta y envía email de confirmación en < 60 s.
CA-02	Email duplicado	El email ingresado ya existe en el sistema	presiona 'Registrarse'	el sistema muestra: 'El correo ya está registrado' sin crear la cuenta.

CA	Escenario	Dado que	Cuando	Entonces
CA-03	Contraseña débil	El usuario ingresa una contraseña de menos de 8 caracteres	escribe en el campo contraseña	el sistema muestra el error en tiempo real sin enviar el formulario.
CA-04	Campos obligatorios vacíos	El usuario deja en blanco nombre o email	presiona 'Registrarse'	el sistema resalta los campos vacíos y no procesa el envío.

5.2 Plantilla de Sprint Backlog

ID HU	Historia de Usuario	Criterios de Aceptación	Estimación (SP)	Prioridad MoSCoW	Sprint
HU-01	Registro de usuario	CA-01 al CA-04	5	Must Have	Sprint 1
HU-02	Inicio de sesión	CA-05 al CA-08	3	Must Have	Sprint 1
HU-03	Listado de productos	CA-09 al CA-11	5	Must Have	Sprint 1
HU-04	Filtros de búsqueda	CA-12 al CA-14	3	Should Have	Sprint 2
HU-05	Perfil de usuario	CA-15 al CA-17	3	Should Have	Sprint 2
HU-06	Modo oscuro	CA-18	1	Could Have	Sprint 3

6. Errores Frecuentes al Redactar Requisitos

Error	Ejemplo incorrecto	Corrección
Ambigüedad	El sistema debe ser rápido.	El sistema debe responder en < 500 ms al 95% de las peticiones.
Múltiples funciones	El sistema registra y envía email y notifica al admin.	Separar en 3 requisitos independientes.
Sin criterio de éxito	El sistema permitirá el login.	Agregar condición verificable: bloqueo tras 3 intentos, JWT, etc.
Solución en el req.	El sistema usará React para mostrar el listado.	El sistema mostrará el listado de productos con filtros. (La tecnología se decide en diseño.)
Sin actor	Se podrán cargar imágenes.	El administrador podrá cargar imágenes en formato JPG/PNG de hasta 2 MB.
No medible (RNF)	El sistema debe ser accesible.	El sistema debe cumplir con las pautas WCAG 2.1 nivel AA.

PARTE II — README.md para Repositorios en GitHub

7. ¿Para qué sirve el README.md?

El archivo README.md es la tarjeta de presentación de un repositorio. Es lo primero que ve cualquier persona que visita el repositorio en GitHub. Un buen README cumple tres funciones:

- Explica QUÉ es el proyecto, para qué sirve y a quién está dirigido.
- Indica CÓMO instalar, configurar y ejecutar el proyecto desde cero.
- Documenta el ESTADO del proyecto: tecnologías, autores, estado de desarrollo, licencia.

En el contexto académico, el README es además un instrumento de evaluación: demuestra que el equipo comprende su propio proyecto y puede comunicarlo con claridad profesional.

8. Secciones Obligatorias del README.md

8.1 Encabezado del proyecto

Markdown — Encabezado

```
# Nombre del Proyecto > Breve descripción del sistema (1-2 oraciones). Qué hace y para
quién. ![Estado](https://img.shields.io/badge/estado-en%20desarrollo-yellow)
![Versión](https://img.shields.io/badge/versión-1.0.0-blue)
![Licencia](https://img.shields.io/badge/licencia-MIT-green)
```

8.2 Tabla de contenidos

Markdown — Tabla de Contenidos

```
## Tabla de Contenidos - [Descripción](#descripción) - [Tecnologías](#tecnologías) -
[Requisitos previos](#requisitos-previos) - [Instalación](#instalación) - [Uso](#uso) -
[Estructura del proyecto](#estructura-del-proyecto) - [Equipo](#equipo) -
[Licencia](#licencia)
```

8.3 Descripción del proyecto

Markdown — Descripción

```
## Descripción **[Nombre del Proyecto]** es una aplicación web desarrollada como
proyecto integrador de la materia **Programación II** — Tecnicatura Superior en Desarrollo
Web. El sistema permite a los usuarios [funcionalidad principal]. Está orientado a [tipo de
usuario/organización]. ### Objetivo del sistema <!-- Relacionar con el problema que
resuelve y el valor que aporta -->
```

8.4 Stack tecnológico

Markdown — Tecnologías

```
## 🛠️ Tecnologías utilizadas | Capa      | Tecnología      | Versión | |-----|-----|
-----|-----| | Backend      | Django          | 5.x      | | API REST    | Django REST
Framework | 3.x      | | Base de datos | PostgreSQL      | 16.x     | | Frontend  | Angular      |
17.x     | | Lenguajes   | Python 3.11 / TypeScript 5 | | Control de versiones | Git / GitHub |
| | Gestor de paquetes | pip / npm      | |
```


8.5 Requisitos previos e Instalación

Markdown — Instalación (Backend Django)

```
## Requisitos previos - Python 3.11 o superior - Node.js 20.x o superior - PostgreSQL 16.x
- Git ## Instalación #### Backend (Django) ```bash # 1. Clonar el repositorio git clone
https://github.com/usuario/nombre-repositorio.git cd nombre-repositorio/backend # 2. Crear
y activar entorno virtual python -m venv venv venv\Scripts\activate # Windows source
venv/bin/activate # Linux / macOS # 3. Instalar dependencias pip install -r
requirements.txt # 4. Configurar variables de entorno cp .env.example .env #
completar las variables # 5. Aplicar migraciones python manage.py migrate # 6. Crear
superusuario (opcional) python manage.py createsuperuser # 7. Ejecutar el servidor
python manage.py runserver ```
```

Markdown — Instalación (Frontend Angular)

```
#### Frontend (Angular) ```bash cd ../frontend # Instalar dependencias npm install #
Ejecutar en modo desarrollo ng serve ``` La aplicación estará disponible en:
http://localhost:4200
```

8.6 Variables de entorno

Markdown — Variables de entorno (.env.example)

```
## Variables de entorno Crear un archivo `.env` en la raíz del backend con los siguientes
valores: ```env SECRET_KEY=tu_clave_secreta_django DEBUG=True
DB_NAME=nombre_base_de_datos DB_USER=postgres DB_PASSWORD=tu_contraseña
DB_HOST=localhost DB_PORT=5432 ALLOWED_HOSTS=localhost,127.0.0.1 ``` > ⚠️
**Nunca subir el archivo `.env` al repositorio.** Agregarlo al `.gitignore`.
```

8.7 Estructura del proyecto

Markdown — Estructura de carpetas

```
## 📁 Estructura del proyecto ``` nombre-repositorio/ |— backend/ | |— my_project/
# Configuración global Django | | |— settings.py | | |— urls.py | |— app/
# Aplicación principal | | |— models.py | | |— views.py | | |— serializers.py
| | |— urls.py | | |— admin.py | | |— requirements.txt | |— manage.py |—
frontend/ | |— src/ | | |— app/ | | |— components/ | | |—
services/ | | |— models/ | | |— angular.json | |— docs/ |— #
Documentación del proyecto | |— requisitos.md | |— diagrama_er.png |—
.gitignore |— README.md ```
```

8.8 Endpoints de la API

Markdown — Documentación de la API

```
## 🌐 Endpoints disponibles Base URL: `http://localhost:8000/api/` | Método | Endpoint
| Descripción | Auth requerida | |-----|-----|-----|-----|-----|-----|
```

```

-----| GET  | /users/      | Listar todos los usuarios  | Sí      | | POST  | /users/
| Crear un nuevo usuario    | No      | | GET  | /users/{id}/ | Obtener usuario por ID
| Sí      | | PUT  | /users/{id}/ | Actualizar usuario completo | Sí      | | DELETE |
/users/{id}/ | Eliminar usuario          | Sí (Admin) | | GET  | /roles/      | Listar roles
disponibles  | Sí      | | POST | /roles/      | Crear un nuevo rol        | Sí (Admin)
|

```

8.9 Estado del proyecto y sprint actual

Markdown — Estado y roadmap

```

## Estado del proyecto | Módulo          | Estado      | |-----|-----| |
Autenticación        | Completado  | | Gestión de usuarios | Completado | | Gestión de roles
| En progreso | | Frontend Angular  | Pendiente  | | Tests unitarios    | Pendiente  |
**Sprint actual:** Sprint 2 — Fecha de cierre: DD/MM/AAAA

```

8.10 Equipo de desarrollo

Markdown — Equipo

```

## 👥 Equipo | Nombre completo    | Rol en el proyecto      | GitHub          | |---
-----|-----|-----| | Apellido, Nombre 1  |
Desarrollador/a Backend | [@usuario1](https://github.com) | | Apellido, Nombre 2  |
Desarrollador/a Frontend | [@usuario2](https://github.com) | | Apellido, Nombre 3  | Líder
técnico / Full Stack | [@usuario3](https://github.com) | **Docente:** [Nombre del Docente]
**Institución:** Tecnicatura Superior en Desarrollo Web y Aplicaciones Móviles
**Materia:** Programación II — Año: 2025

```

8.11 Licencia y contribuciones

Markdown — Licencia

```

## 📄 Licencia Este proyecto fue desarrollado con fines académicos. Distribuido bajo
licencia [MIT](LICENSE). ## Contribuciones. Este es un proyecto académico. Para reportar
errores o sugerencias, abrir un [Issue](https://github.com/usuario/repo/issues) en el
repositorio.

```

9. Buenas Prácticas en el Repositorio GitHub

9.1. gitignore esencial para Django + Angular

.gitignore recomendado

```

# Python / Django venv/ __pycache__/ *.pyc *.pyo *.pyd db.sqlite3 .env *.log media/ #
Node / Angular node_modules/ dist/ .angular/ *.env.local # IDEs .vscode/ .idea/ *.DS_Store

```

9.2 Convención de nombres para ramas (branches)

Tipo de rama	Convención	Ejemplo
Principal	main	main
Desarrollo	develop	develop
Nueva función	feature/HU-ID-descripcion	feature/HU-01-registro-usuario
Corrección bug	fix/descripcion-del-bug	fix/error-login-token
Hotfix urgente	hotfix/descripcion	hotfix/csrf-token-missing
Documentación	docs/descripcion	docs/actualizar-readme

9.3 Convención de mensajes de commit

Formato de Conventional Commits

tipo(ámbito): descripción breve en imperativo Tipos válidos: feat → Nueva funcionalidad fix → Corrección de error docs → Cambios en documentación style → Formato (no afecta lógica) refactor → Mejora de código sin cambiar comportamiento test → Agrega o modifica tests chore → Tareas de mantenimiento Ejemplos: feat(auth): agregar endpoint de registro de usuario fix(login): corregir bloqueo tras intentos fallidos docs(readme): agregar sección de instalación frontend test(users): agregar prueba unitaria para serializer

10. Checklist de Entrega del Proyecto

Antes de presentar el repositorio, verificar que se cumplan todos los puntos:

Ítem	Descripción	Cumple
README.md completo	Incluye todas las secciones obligatorias (secciones 8.1 a 8.11).	☐
Requisitos documentados	El repo incluye un archivo docs/requisitos.md con RF y RNF redactados según IEEE.	☐
Historias de usuario	Cada RF tiene su HU con criterios de aceptación	☐
Sprint backlog	La pestaña Projects / Issues de GitHub refleja el estado de cada HU.	☐
.gitignore configurado	.env, node_modules/, venv/ y MySQL/MariaDB no están en el repo.	☐
requirements.txt	El archivo existe y está actualizado con todas las dependencias.	☐
Variables de entorno	Existe .env.example con las variables necesarias sin valores reales.	☐
Código organizado	La estructura de carpetas sigue la convención documentada.	☐
Commits descriptivos	Los mensajes de commit siguen Conventional Commits.	☐

Ítem	Descripción	Cumple
Ramas por funcionalidad	Cada HU se desarrolló en su propia rama y se mergeó a develop.	☐
API documentada	La sección de endpoints lista todos los recursos disponibles.	☐
Instrucciones funcionales	Otro desarrollador puede clonar y ejecutar el proyecto siguiendo el README.	☐

Recuerda

Un buen README y una buena documentación de requisitos no son burocracia: son la diferencia entre un proyecto amateur y uno profesional. En la industria, la documentación es tan importante como el código que la acompaña.